

**Correction TP n°3 :
Approximation Polynomiale et Optimisation**

Module: Analyse Numérique

Classes: 3A-B

Année universitaire: 2025-2026

Objectif du TP

L'objectif du TP est de :

- Comprendre la démarche d'approximation polynomiale par la méthode des moindres carrés à partir d'un jeu de données.
- Identifier les limites de la méthode analytique.
- Introduire la méthode de descente de gradient comme alternative numérique pour l'estimation des coefficients polynomiaux.
- Visualiser et interpréter graphiquement les résultats obtenus afin de comparer les approches analytiques et numériques.

1 Préparation de l'environnement

Afin d'implémenter les méthodes d'approximation polynomiale et de descente de gradient, nous aurons besoin des bibliothèques `numpy` et `matplotlib.pyplot`.

La bibliothèque `numpy` offre un module dédié à la manipulation des polynômes : `numpy.polynomial`. Celui-ci contient la classe `Polynomial`, qui permet de créer, évaluer, manipuler et même ajuster des polynômes à partir de données expérimentales.

Pour commencer, il faut importer les modules nécessaires en exécutant les commandes suivantes:

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial import Polynomial
```

2 Principe de la méthode des moindres carrés

L'objectif de l'approximation polynomiale est de trouver un polynôme de degré p

$$P(x) = \lambda_0 + \lambda_1 x + \dots + \lambda_p x^p,$$

qui s'ajuste au mieux aux données expérimentales $(x_i, y_i)_{0 \leq i \leq n}$, dont le vecteur des coefficients est noté $\mathbf{\Lambda} = (\lambda_0, \lambda_1, \dots, \lambda_p)^T \in \mathbb{R}^{p+1}$.

Le **principe des moindres carrés** consiste à minimiser la somme des carrés des écarts entre les valeurs observées y_i et les valeurs prédites $P(x_i)$.

Autrement dit, on cherche les coefficients $\mathbf{\Lambda} = (\lambda_0, \lambda_1, \dots, \lambda_p)^T$ qui minimisent la fonction coût $J(\mathbf{\Lambda})$ tel que :

$$\min_{\lambda_0, \lambda_1, \dots, \lambda_p} J(\mathbf{\Lambda}) = \sum_{i=0}^n (y_i - P(x_i))^2 = \|A\mathbf{\Lambda} - Y\|_2^2$$

avec

$$A = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^p \\ 1 & x_1 & x_1^2 & \dots & x_1^p \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^p \end{bmatrix}, \quad \mathbf{\Lambda} = \begin{bmatrix} \lambda_0 \\ \lambda_1 \\ \vdots \\ \lambda_p \end{bmatrix}, \quad Y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix},$$

Au minimum, le gradient de $J(\mathbf{\Lambda})$ donné par $\nabla J(\mathbf{\Lambda}) = 2A^T(A\mathbf{\Lambda} - Y)$, s'annule :

$$A^T(A\mathbf{\Lambda} - Y) = 0_{\mathbb{R}^{p+1}} \implies A^T A \mathbf{\Lambda} = A^T Y.$$

Si la matrice $A^T A$ est inversible, alors la solution du problème de minimisation s'exprime explicitement à l'aide de la formule suivante, appelée *l'équation normale* de la méthode des moindres carrés.

$$\mathbf{\Lambda} = (A^T A)^{-1} A^T Y$$

3 Construction de la matrice du modèle

Question 1 :

Écrire une fonction nommée `matrice(X,p)` prenant en entrée $X = (x_0, x_1, \dots, x_n)^T$, un vecteur colonne des abscisses, et un ordre du modèle polynomial p , et retourne la matrice A définie par :

$$A = \begin{pmatrix} 1 & x_0 & x_0^2 & \dots & \dots & x_0^p \\ 1 & x_1 & x_1^2 & \dots & \dots & x_1^p \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \dots & \dots & x_n^p \end{pmatrix} \in \mathcal{M}_{n,p}(\mathbb{R}).$$

```
[ ]: def matrice(X,p):
    n = len(X)
    A = np.zeros((n,p+1))
    for j in np.arange(0,p+1):
        A[:,j] = X[:,0] **j
    return A
```

Question 2 :

Écrire une fonction nommée `regression(X,Y,p)`, prenant en entrée $X = (x_0, x_1, \dots, x_n)^T$ un vecteur colonne des abscisses $Y = (y_0, y_1, \dots, y_n)^T$ un vecteur colonne des ordonnées, p un ordre

du modèle polynomial, et retourne le vecteur Λ des coefficients estimés du modèle au sens des moindres carrés vérifiant:

$$\Lambda = ({}^tAA)^{-1} {}^tAY.$$

```
[ ]: def regression(X,Y,p):
    A= matrice(X,p)
    Lambda= np.linalg.inv(np.transpose(A).dot(A)).dot(np.transpose(A)).dot(Y[:,0])
    return Lambda
```

4 Application numérique : droite de régression

Question 3 :

Déterminer l'expression de la droite de régression à partir le jeu de données $(x_0, y_0) = (-1, 2)$, $(x_1, y_1) = (0, 1)$ et $(x_2, y_2) = (1, -1)$.

```
[ ]: # Données expérimentales
X = np.array([-1,0,1]).reshape(-1, 1)
Y = np.array([2,1,-1]).reshape(-1, 1)

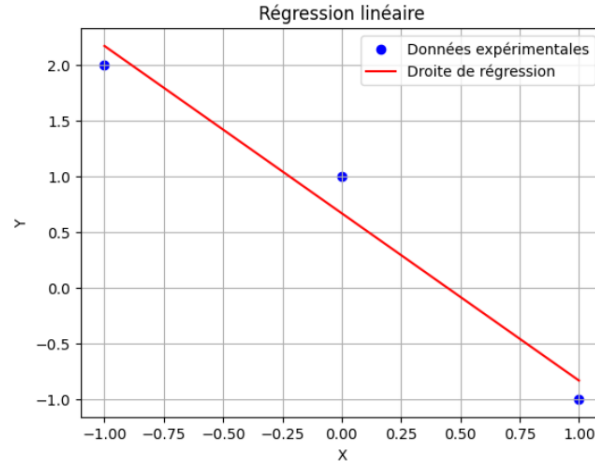
p=1
Lambda=regression(X, Y, p)
Polynomial(Lambda)
```

$x \mapsto 0.666 - 1.5x$.

Question 4 :

Représenter sur la même figure la droite de régression, ainsi que le jeu de données.

```
[ ]: # Visualisation
plt.scatter(X, Y, color='blue', label='Données expérimentales')
plt.plot(X, Polynomial(Lambda)(X), color='red', label='Droite de régression')
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Régression linéaire")
plt.legend()
plt.grid(True)
plt.show()
```



5 Limites de la méthode analytique

La solution obtenue par l'équation normale

$$\Lambda = (A^T A)^{-1} A^T y$$

présente l'avantage d'être explicite et simple à implémenter pour des jeux de données de taille modérée. Cependant, cette approche soulève plusieurs limites théoriques et numériques importantes.

Limitations	Description et impact
Coût de calcul élevé	Le calcul de l'inverse de la matrice $A^T A$ nécessite une complexité de l'ordre de $O(p^3)$. Pour des modèles polynomiaux de degré élevé ou des bases de grande dimension, ce coût devient prohibitif en temps et en mémoire.
Instabilité numérique	Lorsque les colonnes de la matrice A sont fortement corrélées, la matrice $A^T A$ devient mal conditionnée. De petites perturbations dans les données X ou Y peuvent alors induire de grandes variations dans les coefficients estimés Λ . Dans les cas extrêmes, $A^T A$ peut être singulière et non inversible.
Sensibilité aux erreurs d'arrondi	Les erreurs de calcul dues à la précision machine (arrondis numériques) sont amplifiées lorsque le conditionnement est mauvais, ce qui dégrade la précision du modèle ajusté.
Manque de flexibilité	La méthode analytique est spécifique à une fonction de coût quadratique. Elle ne s'adapte pas facilement à d'autres fonctions d'erreur, à des contraintes sur les coefficients, ou à des modèles plus complexes (non linéaires, réseaux de neurones, etc.).

Face à ces limitations, on privilégie souvent des **méthodes itératives** telles que la **descente de gradient**. Cette approche permet d'éviter le calcul explicite de l'inverse, de mieux contrôler la

précision du résultat, et d'être applicable à une grande variété de fonctions de coût. L'idée principale est de partir d'une estimation initiale $\Lambda^{(0)}$ et de la mettre à jour progressivement dans la direction opposée au gradient de la fonction objectif :

$$\begin{aligned}\Lambda^{(k+1)} &= \Lambda^{(k)} - \eta \nabla J(\Lambda^{(k)}) \\ &= \Lambda^{(k)} - 2\eta A^T (A\Lambda^{(k)} - Y)\end{aligned}$$

où $\eta > 0$ est le *pas d'apprentissage*.

L'algorithme itératif s'arrête lorsque l'un des critères suivants est satisfait :

- la norme du gradient devient suffisamment petite :

$$\|\nabla J(\Lambda^{(k)})\|_2 < \varepsilon,$$

où $\varepsilon > 0$ est une tolérance fixée à l'avance.

- le nombre maximal d'itérations $N_{\max}$ est atteint.

La section suivante est consacrée à l'implémentation numérique du *schéma itératif de la descente de gradient*.

6 Méthode de la descente de gradient

Question 5 :

Écrire une fonction nommée `gradient_descent` permettant d'estimer les coefficients d'un polynôme de degré p en utilisant la méthode de descente de gradient. La fonction doit prendre en entrée les vecteurs X et Y , le degré p , un vecteur initial `Lambda0`, un pas d'apprentissage `eta`, un critère d'arrêt `epsilon` basé sur la norme du gradient et un nombre maximal d'itérations `max_iter`. La fonction doit retourner le vecteur des coefficients optimisés et l'historique des valeurs de la fonction coût et de son gradient.

```
[ ]: def gradient_descent(X, Y, p, Lambda0, eta, epsilon, max_iter):

    # Construire la matrice du modèle
    A = matrice(X, p)

    # Initialisation des coefficients et de l'historique
    Lambda = Lambda0.copy()
    J_hist = []
    Grad_hist = []

    for k in range(max_iter):
        # Calculer le gradient
        gradient = 2*(A.T .dot(A.dot(Lambda) - Y[:,0]))
        Grad_hist.append(gradient)

        # Mettre à jour les coefficients
```

```

Lambda = Lambda - eta * gradient

# Calculer la fonction coût
J = np.linalg.norm(A.dot(Lambda) - Y[:,0])**2
J_hist.append(J)

# Critère d'arrêt
if np.linalg.norm(gradient) < epsilon:
    break

return Lambda, J_hist, Grad_hist

```

Question 6 :

Tester la fonction `gradient_descent` sur les données donné ci-dessus, en choisissant :

$$\Lambda^{(0)} = 0, \quad p = 1, \quad \eta = 10^{-3}, \quad \varepsilon = 10^{-4}, \quad N_{\max} = 10^4.$$

```

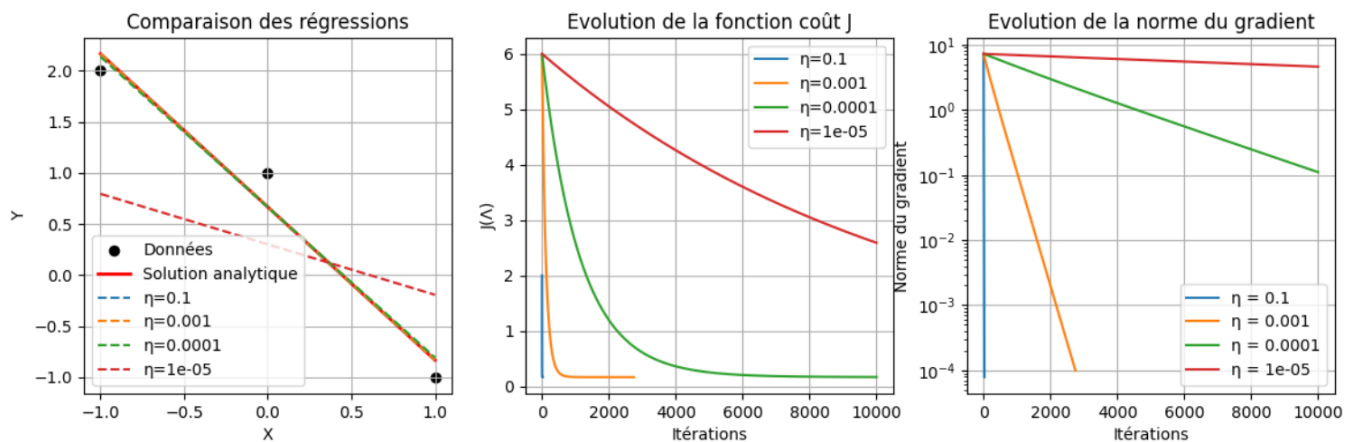
[ ]: Lambda0=np.zeros(p+1)
eta=1e-3
epsilon=1e-4
max_iter=10**4
Lambda_app, J_hist, Grad_hist = gradient_descent(X, Y, p, Lambda0, eta, epsilon, max_iter)
Polynomial(Lambda_app)

```

$x \mapsto 0.66666662 - 1.4999752x$.

Question 7 :

En observant les graphiques obtenus (droites de régression, évolution de la fonction coût et de la norme du gradient) pour différentes valeurs du pas d'apprentissage $\eta \in \{10^{-1}, 10^{-3}, 10^{-4}, 10^{-6}\}$, proposer une valeur optimale de η en justifiant votre choix.



Valeur de η	Comportement de la convergence	Évolution de $J(\Lambda)$	Qualité de la solution	Commentaire
10^{-1}	Rapide mais légèrement instable	Diminution rapide	Proche de la solution analytique	Risque d'instabilité si problème mal conditionné
10^{-3}	Stable et régulière	Décroissance monotone et fluide	Très proche de la solution analytique	Meilleur compromis vitesse/stabilité
10^{-4}	Stable	Diminution progressive et lente	proche de la solution analytique	Convergence stable mais coûteuse
10^{-5}	Très lente	Diminution très lente	Très loin de la solution analytique	Convergence très lente et très coûteuse

7 Limites de la descente de gradient à pas fixe et alternatives numériques

La descente de gradient à pas fixe fonctionne correctement lorsque le pas d'apprentissage est bien choisi et que le problème est bien conditionné. Cependant, elle reste numériquement limitée et peu efficace pour des systèmes mal conditionnés ou de grande dimension.

Ces limites motivent l'utilisation de méthodes plus robustes, telles que la descente de gradient avec pas adaptatif, les méthodes de gradient stochastique (SGD), la méthode du gradient conjugué ou encore les optimiseurs avancés comme Adam, RMSProp et Adagrad, qui combinent pas adaptatif et moyenne des gradients pour stabiliser l'apprentissage. De plus, la normalisation et le prétraitement des données, par centrage ou standardisation, permettent d'améliorer le conditionnement et d'accélérer la convergence. L'introduction de termes de régularisation constitue également un atout majeur : elle permet de contrôler la complexité du modèle et de favoriser une meilleure généralisation des résultats.

Note pédagogique : Ces alternatives sont à étudier dans les modules avancés de Machine Learning, Deep Learning et IA, où les problèmes sont souvent non linéaires et de grande dimension. La descente de gradient à pas fixe reste néanmoins un outil pédagogique essentiel pour comprendre le principe de l'optimisation et la minimisation des fonctions coût.